

Tarea 01

Perlin Peak

Profes: Nancy Hitschfeld Kahler, Valentín Muñoz Apablaza

Auxiliares: Julieta Coloma, Catalina Gajardo

1 - Introducción

En esta tarea programarán un generador de terreno usando **Perlin noise**, se verá como la siguiente imagen:

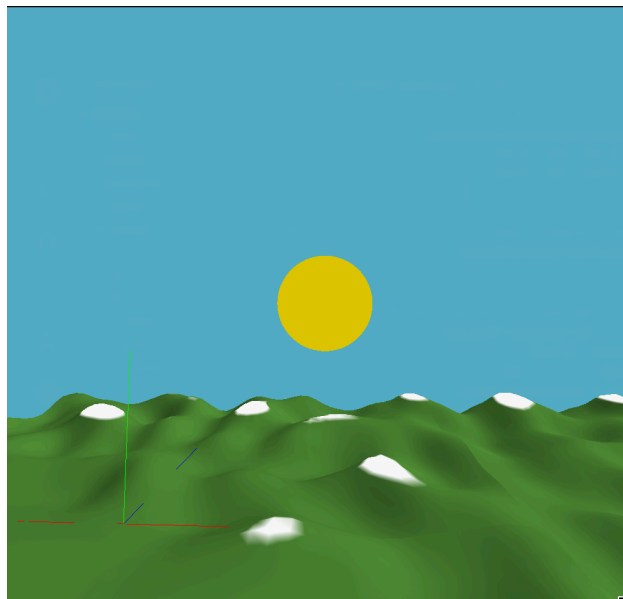


Figura 1: Screenshot de una posible solución a la tarea, también vean este [video](#)

La escena consiste en un terreno compuesto por *chunks* y un sol. En estos chunks se genera una superficie donde la altura de cada punto está dado por Perlin Noise. Además se destacan los puntos más altos con nieve, por medio del fragment shader. Por último, el sol será representado por un círculo que oscila en el eje Y.

1-1 - Chunks

Normalmente, nos referimos a un **chunk** cuando hablamos de un fragmento de datos. En la generación de terreno es común dividir el espacio en chunks, de esta forma se puede optimizar la renderización en función de la cantidad de chunks que se quieren mostrar. Un ejemplo muy conocido de esta técnica es *Minecraft*, donde el mundo se genera a través de chunks de 16×16 bloques.

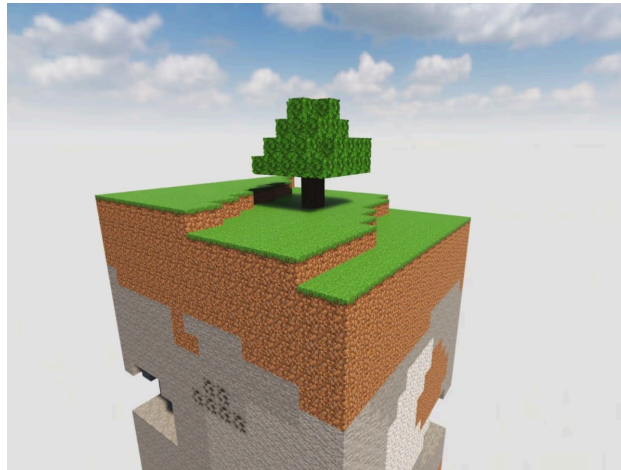


Figura 2: Un chunk de Minecraft

En esta tarea tendrán que trabajar generando chunks de $n \times n$ cuadrados. Cada chunk tendrá dos índices que lo identificarán. La siguiente imagen muestra el chunk $(0, 0)$ y los 8 chunks que lo rodean, visto desde arriba:

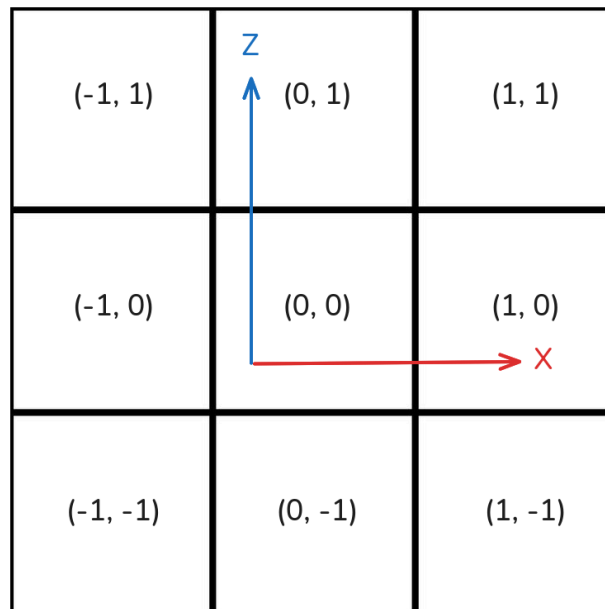
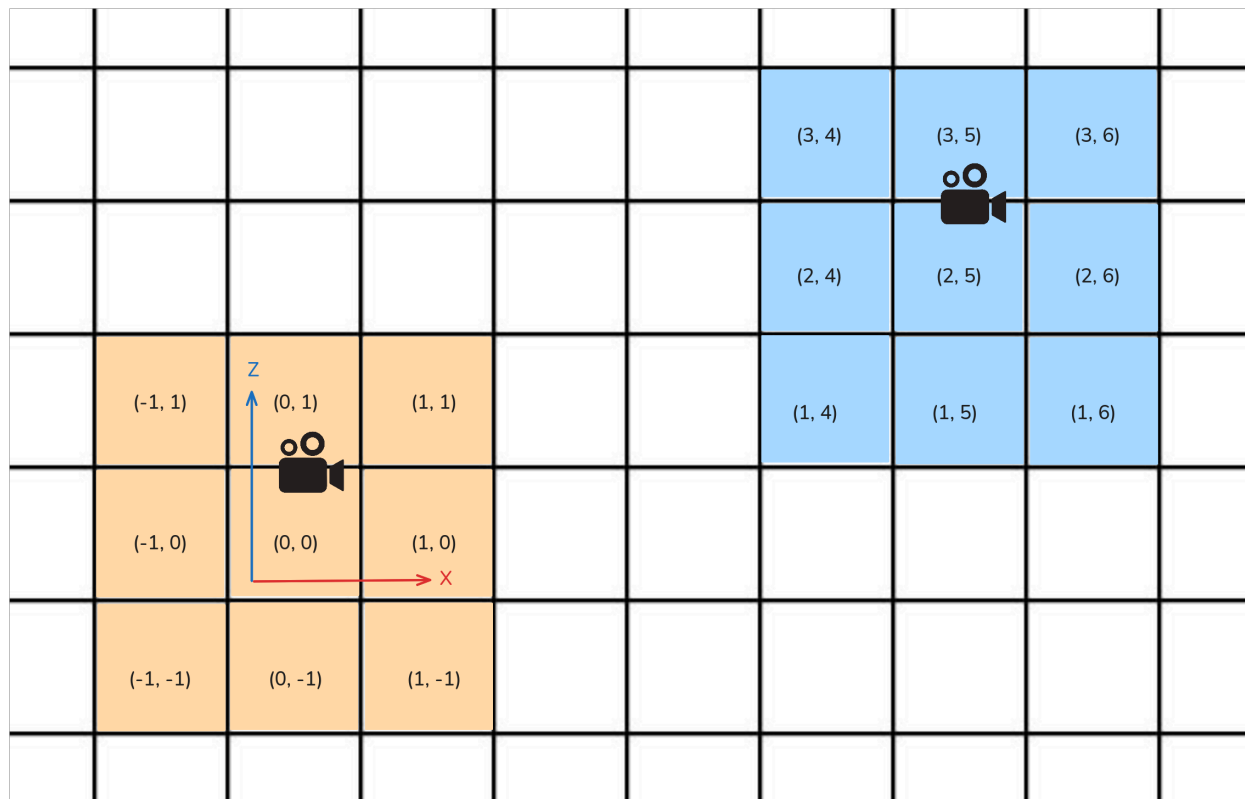


Figura 3: Chunk origen y sus vecinos

Estos identificadores se usarán para llevar una lista de los chunks actualmente activos. Para determinar cuáles chunks estarán activos se utilizará la posición de la cámara: estarán activos el chunk que está directamente bajo la cámara y los 8 que lo rodean. Observen estos dos ejemplos para mayor claridad, donde se muestran los chunks activos si la cámara esta en 0.0 (azul) o si la cámara está en el chunk $(2, 5)$:



1-2 - Perlin Noise

El [Perlin noise](#) es un tipo de ruido altamente usado para la generación de terreno. Entendemos ruido como valores al azar, que nos permiten darle un aspecto más natural a nuestro terreno.

En esta tarea, utilizarán una clase del módulo `perlin_noise`. Para usarla, primero se crea un objeto `noise`:

```
noise = PerlinNoise(octaves=4, seed=12)
```

py

Y luego, se puede obtener un valor en función de la coordenada del terreno. Por ejemplo, si quisieran obtener la altura del terreno para el triángulo en $(x=20, z=35)$, tendrían que hacer lo siguiente:

```
x = 20.0
z = 35.0
altura = noise(x, z)
punto = [x, altura, z]
```

py

1-3 - El sol

El sol será simplemente un círculo que se encontrará a una distancia apropiada del centro y que estará en un movimiento periódico de arriba hacia abajo. *Recuerden que vimos como hacer círculos en un aux....*

1-4 - Template

Junto a este enunciado se les entrega una carpeta que contiene el esqueleto de su proyecto. La mayor parte del trabajo la deben realizar en el módulo `terrain.py` donde completarán las funciones. Pero también deben completar el programa principal `tarea1.py` y los shaders. Está claramente indicado con **comentarios** donde falta completar en los archivos.

2 - Trabajo a realizar

Su tarea **debe** incluir lo siguiente:

1. Generación de la geometría del Sol:

- Implementar `createSunVertices(x, y, radius, DEFINITION)` para calcular las coordenadas de un círculo en el plano XY junto a su centro.
- Implementar `createSunIndices(DEFINITION)` para generar el arreglo de índices conectando los triángulos del círculo.
- Instanciar la geometría del sol en `main.py`, mediante un `vertex_list_indexed` asociado al pipeline correspondiente.

2. Generación de terreno con Perlin Noise:

- Implementar `generateChunkVertices(n, chunk_x, chunk_z)`: debe construir la lista de vértices e índices para un chunk de tamaño $n \times n$ ubicado en las coordenadas de chunk indicadas, evaluando las alturas Y con la librería `PerlinNoise`. Debe calcular y adjuntar las normales de la superficie utilizando la función provista `calculateNormals()`.
- Implementar `makeChunks(resolucion, controller, pipeline)`: construye los `vertex_list` de cada chunk activo y los almacena en `controller.gpu_chunks`.
- Implementar `updateChunks(resolucion, camara, controller, pipeline)`: monitorea la posición de la cámara (X, Z) y actualiza dinámicamente los índices de `controller.currentChunks` para instanciar nuevos chunks según el movimiento del usuario.

3. Shaders y renderizado:

- **Fragment Shaders**: Definir adecuadamente las variables de entrada (`in`) y salida (`outColor`) en los archivos `.frag` para recibir los datos de posición y normal interpolados desde el vertex shader.
- **Render Loop**: Configurar y pasar las matrices y uniforms necesarios a través de los pipelines del sol y del terreno.
- **Animación**: En la función `update()`, aplicar una transformación u oscilación periódica temporal en el eje Y al sol.

2 -1 - Material Complementario

- **Auxiliares**: Vean el material entregado en el repositorio tanto de este año como años anteriores.
- **Template de la tarea**: Revisar la estructura base entregada en `main.py`, `terrain.py` y los archivos GLSL.

3 - Bonus

Se otorgará un bono de hasta 0.5 puntos adicionales en alguna tarea (acumulable para las futuras) si programa **uno** de los siguientes elementos:

- **Sol Orbital con Distancia Fija a la Cámara**: Modificar la matriz de transformación del sol para que mantenga una distancia constante respecto a la cámara mientras orbita en torno al terreno.
- **Optimización de Chunks**: Al moverse de un chunk a otro, se generan todos los chunks que rodean al central de nuevo. Sin embargo hay algunos que se repiten. Implemente una optimización sobre este proceso para generar solo los chunks nuevos.

4 - Entregable

Se debe adjuntar un archivo `.zip` que contenga todos los elementos necesarios para ejecutar la tarea correctamente. Esto incluye, imágenes requeridas, módulos utilizados, etc.

Se aplicarán descuentos en la calificación en los siguientes casos:

- -0.3 puntos por nombres mal escritos.
- -0.3 puntos por errores en las rutas o caminos de los archivos.
- -0.5 puntos por el uso de librerías no incluidas en el curso sin previo aviso.
- -0.5 puntos por la entrega con archivos faltantes.

Si su tarea no puede ser ejecutada por alguna otra razón quedará a criterio del equipo docente evaluarla con la posibilidad de nota mínima si es que lo amerita.

Recomendaciones:

1. Descargar el archivo .zip y verificar que el programa se ejecuta debidamente.
2. Incluir un archivo README.txt en caso de que la ejecución de la tarea requiera pasos adicionales.

5 - Consideraciones

Debe tener en cuenta lo siguiente:

- El plazo de entrega es inamovible.
- El trabajo es individual.
- No está permitido el plagio del trabajo de sus compañeros.
- Es obligatoria la entrega de la declaración de uso de IA.